

The Qubit Hamiltonian

Second quantisation, fermion-to-qubit mappings, and the stage every tutorial skips

Phase 2 builds the translation layer between a molecule and a quantum computer. It is the stage most VQE demonstrations bypass by hardcoding Pauli coefficients — and bypassing it is what makes those demonstrations unable to change geometry, basis, or mapping.

By the end you will convert a bond length into a qubit operator, verify that operator against Phase 1's exact result, and reduce four qubits to one using nothing but the symmetries of the Hamiltonian.

Estimated time: **2-3 hours**.

In this document

Why encoding states is easy and encoding operators is not

Second quantisation: creation and annihilation operators, and where the integrals come from

A runnable eight-line demonstration of why the naive qubit encoding is wrong

Jordan-Wigner, parity, and the locality trade-off between them

Symmetry tapering: four qubits to one, with the energy unchanged

The verification against Phase 1 that gates the rest of the project

1. Where this phase sits

This is the translation layer: molecule in, qubit operator out. It is the stage that almost every VQE tutorial skips by handing you a list of hardcoded Pauli coefficients, and it is the single largest capability gain in this roadmap.

Why this phase deserves its own document

A hardcoded Hamiltonian cannot change geometry — which makes Phase 4, the entire point of the project, impossible.

It also silently locks in somebody else's basis set, somebody else's fermion-to-qubit mapping, somebody else's tapering, and somebody else's convention about whether nuclear repulsion is included. You inherit four decisions and have a handle on none of them.

Build this stage properly once and every other molecule, basis and mapping becomes a one-line change.

2. Theory

2.1 Encoding the states is the easy half

Phase 1 established that H_2 in STO-3G has four spin orbitals. Encoding *states* is then as obvious as you would hope: one qubit per spin orbital, $|1\rangle$ for occupied and $|0\rangle$ for empty.

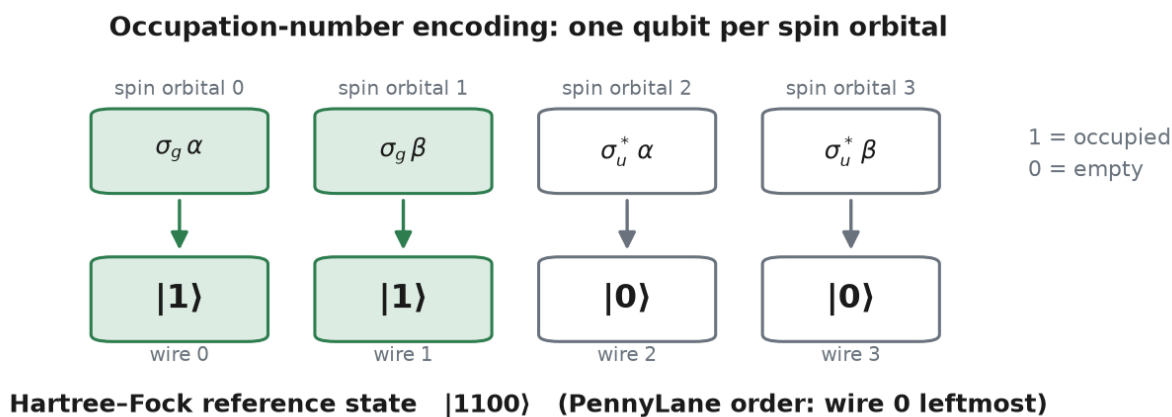


Figure 2.1 — Occupation-number encoding. The Hartree-Fock ground state from Phase 1 becomes the four-bit string $|1100\rangle$.

That part needs no cleverness at all. **The problem is that the Hamiltonian is not written in terms of states.**

2.2 Second quantisation

The Hamiltonian is written in terms of operators that move electrons between orbitals:

$$\hat{H} = \sum_{pq} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} h_{pqrs} a_p^\dagger a_q^\dagger a_r a_s$$

$a_p^\dagger$	<i>Creation operator.</i> Puts an electron into spin orbital p . Read aloud as "a-dagger-p".
a_q	<i>Annihilation operator.</i> Removes an electron from spin orbital q .
h_{pq}	<i>One-electron integrals.</i> Plain numbers describing an electron hopping from orbital q to p — kinetic energy plus nuclear attraction. PySCF computes these from your basis set and geometry.
h_{pqrs}	<i>Two-electron integrals.</i> The electron–electron repulsion, scattering two electrons from orbitals r,s into p,q . There are $O(N^4)$ of them, which is why basis-set size hurts.

So building a qubit Hamiltonian does not mean translating states. It means translating **$a^\dagger$ and a** . That is what a mapping is.

2.3 The rule qubits do not obey

Electrons are fermions, and fermions carry a hard constraint:

$$a_p^\dagger a_q^\dagger = -a_q^\dagger a_p^\dagger$$

Create two electrons in the opposite order and the wavefunction changes sign. This is antisymmetry, and the Pauli exclusion principle is just the $p = q$ case of it: $a_p^\dagger a_p^\dagger = -a_p^\dagger a_p^\dagger$ forces that product to be zero, so no orbital takes two electrons.

Qubits have no such rule. Qubits are distinguishable and sit on separate wires, so operators acting on different qubits simply **commute**. The obvious guess — " $a_p^\dagger$ = raise qubit p " — is therefore broken before you start.

2.4 Watch it break

This is worth running yourself rather than taking on trust. It is eight lines and it makes the entire need for a mapping concrete.

Listing 2.1 — anticommute demo.py

```
"""anticommute_demo.py - why the naive qubit encoding fails."""

import numpy as np

I = np.eye(2)
Z = np.diag([1, -1]).astype(complex)
sp = np.array([[0, 0], [1, 0]], dtype=complex) # |1><0| : creates an electron
k = np.kron

# --- NAIVE: a_p^dag = sigma^+ on qubit p, and nothing else ---
n0 = k(I, sp) # qubit 0
n1 = k(sp, I) # qubit 1
print("NAIVE a0+a1 == a1+a0 ?", np.allclose(n0 @ n1, n1 @ n0),
      " <- they COMMUTE: wrong for fermions")

# --- JORDAN-WIGNER: a_p^dag = (Z on every qubit below p) * sigma^+ on p ---
j0 = k(I, sp) # p=0: nothing below, so no string
j1 = k(sp, Z) # p=1: Z on qubit 0, sigma^+ on qubit 1
print("JW a0+a1 == -a1+a0 ?", np.allclose(j0 @ j1, -(j1 @ j0)),
      " <- they ANTICOMMUTE: correct")

print("because Z and sigma^+ on the SAME qubit anticommute:",
```

```
np.allclose(Z @ sp, -sp @ Z))
```

```
NAIVE a0+a1+ == a1+a0+ ? True  <- they COMMUTE: wrong for fermions
JW    a0+a1+ == -a1+a0+ ? True  <- they ANTICOMMUTE: correct
because Z and sigma^+ on the SAME qubit anticommute: True
```

The naive encoding gets every *state* right and every *sign* wrong. Since the Hamiltonian is built entirely from these operators, wrong signs means a wrong Hamiltonian and a wrong energy.

2.5 Jordan-Wigner

The fix attaches a string of Z operators:

$$a_p^\dagger = (Z_0 Z_1 \cdots Z_{p-1}) \otimes \sigma_p^+$$

The bracketed factor is called the **Jordan-Wigner string**.

Jordan-Wigner: the Z string pays a sign for every electron you hop over

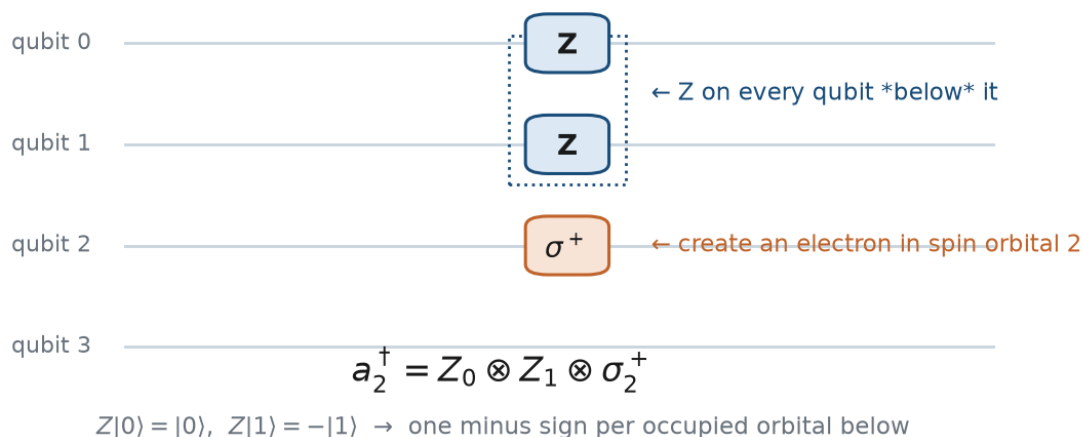


Figure 2.2 — The Z string runs over every qubit below the one being acted on.

Since $Z|0\rangle = |0\rangle$ and $Z|1\rangle = -|1\rangle$, the string contributes a minus sign for each **occupied** orbital below p . Physically: line the orbitals up in a fixed order, and every time you insert or remove an electron, count how many electrons you had to pass on the way and pay a sign per crossing. That bookkeeping *is* antisymmetry.

The mechanism, in one line — and it is the last line of Listing 2.1 — is that Z and σ^+ **on the same qubit** anticommute. When you commute $a_1^\dagger$ past $a_0^\dagger$, the Z in $a_1^\dagger$'s string lands on qubit 0 where $a_0^\dagger$'s σ^+ lives, and produces exactly the -1 required.

The cost: locality is destroyed

An operator on orbital p becomes a Pauli string touching up to p qubits. A physically local process — one electron hopping — turns into a highly non-local qubit operation, which on hardware means long chains of two-qubit gates.

The trade-off in a phrase: **Jordan-Wigner makes occupation local and parity non-local.** Qubit p tells you directly whether orbital p is occupied; finding out how many are occupied below p means reading everything.

2.6 Parity mapping — the mirror image

Parity mapping stores the running total instead — qubit p holds

$$(n_0 + n_1 + \dots + n_p) \bmod 2$$

Two encodings of the same four spin orbitals

	JORDAN-WIGNER qubit p stores n_p	PARITY qubit p stores $n_0 + \dots + n_p \bmod 2$
$n_0 = 1$	1	1 = (1) mod 2
$n_1 = 1$	1	0 = (1+1) mod 2
$n_2 = 0$	0	0 = (1+1+0) mod 2
$n_3 = 1$	1	1 = (1+1+0+1) mod 2
	occupation local · parity non-local	parity local · occupation non-local

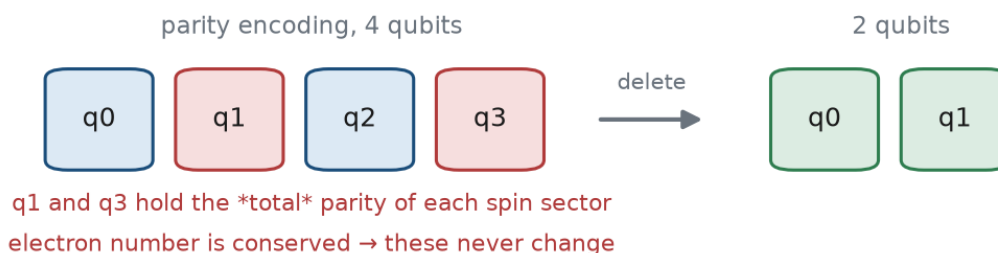
Figure 2.3 — The same four occupations under both encodings. Each makes one thing local and the other non-local.

Now parity is local — just read qubit p — and occupation is non-local, since n_p is qubit p XOR qubit $p-1$. The strings flip from Z's below to X's above, and the asymptotic cost is the same. The two mappings are duals of one another.

2.7 Why parity earns its keep: qubit reduction

In the parity encoding, the qubit for the *last* spin orbital of each spin sector holds the total parity of that sector — whether the number of spin-up electrons is even or odd, and likewise for spin-down. But the Hamiltonian **conserves** both of those. Electron count does not change. So those qubits never do anything: they are constants fixed by how many electrons you have, and can be deleted and replaced by ± 1 by hand.

Why the two-qubit reduction works



This is the step that produced the 5-term, 2-qubit H_2 Hamiltonian you see in tutorials.

*It is a **choice**, not a law — and hardcoding its output is how conventions get silently inherited.*

Figure 2.4 — The two-qubit reduction.

Where the tutorial Hamiltonian comes from

This is exactly the step that produces the five-term, two-qubit H_2 operator you see in textbooks and IBM tutorials. H_2 has further Z_2 symmetries you can exploit — as you will verify in Step 2.5, the full tapering takes it all the way down to **one qubit with three Pauli terms**.

Bravyi-Kitaev is the third common option: a compromise in which both occupation and parity cost $O(\log N)$ rather than one being free and the other $O(N)$. Best asymptotic scaling, hardest to reason about, rarely used in tutorials.

3. Build guide

Step 2.1 — Geometry in, qubit Hamiltonian out

Create `hamiltonian.py`. The one function that matters takes a bond length and returns an operator.

Listing 2.2 — `hamiltonian.py`

```
"""hamiltonian.py - build the qubit Hamiltonian for H2 at any geometry."""

import numpy as np
import pennylane as qml

BOHR_PER_ANGSTROM = 1.8897261254578281
SYMBOLS = ["H", "H"]
ELECTRONS = 2

def qubit_hamiltonian(r_angstrom, basis="sto-3g", mapping="jordan_wigner"):
    """Return (H, n_qubits) for H2 with the nuclei r_angstrom apart.

    PennyLane's qchem module expects coordinates in BOHR, not Angstrom.
    Getting this wrong is the single most expensive bug in the project,
    so the conversion is done explicitly and in one place.
    """
    r_bohr = r_angstrom * BOHR_PER_ANGSTROM

    coordinates = np.array([
        0.0, 0.0, 0.0,      # H atom 1 at the origin
        0.0, 0.0, r_bohr,   # H atom 2 along z
    ])

    H, n_qubits = qml.qchem.molecular_hamiltonian(
        SYMBOLS,
        coordinates,
        basis=basis,
        mapping=mapping,
    )
    return H, n_qubits

if __name__ == "__main__":
    H, n = qubit_hamiltonian(0.735)
    coeffs, ops = H.terms()
    print(f"qubits      {n}")
    print(f"Pauli terms {len(coeffs)}")
    print()
    for c, op in zip(coeffs, ops):
        print(f" {c:+.8f}  {op}")
```

Bohr versus Ångström — read this before running anything

PennyLane's `qchem` coordinates default to **atomic units (Bohr)**. PySCF's `gto.M` defaults to **Ångström**. You will use both in this project.

Symptom: the minimum appears at a physically absurd distance, or the curve has no minimum at all. If 0.735 is passed as Bohr it is really 0.389 Å, deep inside the repulsive wall.

Fix: convert explicitly, in exactly one place, as Listing 2.2 does. Check this first whenever anything looks wrong.

1 Å = 1.8897261 Bohr.

Step 2.2 — Look at what you built

Abbreviated output

```
$ python hamiltonian.py
qubits      4
Pauli terms 15

-0.09057899  I()
+0.17218393  Z(0)
+0.17218393  Z(1)
+0.16892754  Z(0) @ Z(1)
-0.22575349  Z(2)
+0.12091263  Z(0) @ Z(2)
...
```

Fifteen terms on four qubits. Compare that with a hardcoded two-qubit Hamiltonian's five terms and you can see immediately that the tutorial version is a *reduced* operator, not the raw one — a fact that is invisible when the coefficients are just typed in.

Step 2.3 — Verify against Phase 1

This is the critical check of the whole phase. If the qubit Hamiltonian is correct, exactly diagonalising it must reproduce the FCI energy from Phase 1. Nothing in Phase 3 is worth attempting until this passes.

Listing 2.3 — the verification that gates Phase 3

```
import numpy as np
import pennylane as qml
from classical import classical_energies
from hamiltonian import qubit_hamiltonian, ELECTRONS

def verify_against_fci(r_angstrom=0.735):
    """Exact-diagonalise the qubit Hamiltonian and compare with PySCF FCI."""
    _, e_fci, _ = classical_energies(r_angstrom)

    H, n = qubit_hamiltonian(r_angstrom)
    matrix = qml.matrix(H, wire_order=range(n))

    # The qubit Hamiltonian spans ALL electron numbers, 0 through n.
    # Project onto the 2-electron sector to get the chemical ground state.
    occupation = np.array([bin(k).count("1") for k in range(2 ** n)])
    sector = np.where(occupation == ELECTRONS)[0]
    sub = matrix[np.ix_(sector, sector)]
```



```

e_sector = np.linalg.eigvalsh(sub)[0].real
e_global = np.linalg.eigvalsh(matrix)[0].real

print(f"PySCF FCI                      {e_fci:12.8f} Ha")
print(f"qubit H, 2-electron sector    {e_sector:12.8f} Ha")
print(f"qubit H, all sectors            {e_global:12.8f} Ha")
print(f"difference                      {abs(e_sector - e_fci):12.2e} Ha")

assert abs(e_sector - e_fci) < 1e-8, "qubit Hamiltonian does NOT match FCI"
print("\nqubit Hamiltonian verified against FCI")

if __name__ == "__main__":
    verify_against_fci()

```

```

PySCF FCI                      -1.13730604 Ha
qubit H, 2-electron sector    -1.13730604 Ha
qubit H, all sectors          -1.13730604 Ha
difference                    2.08e-10 Ha

```

```
qubit Hamiltonian verified against FCI
```

The residual of 2×10^{-10} Ha is not numerical noise in the diagonalisation — it is the difference between the CODATA Bohr-to-Ångström conversion PySCF uses internally and the one you wrote in `hamiltonian.py`. Anything below about 10^{-8} Ha here is conversion-constant drift, not a physics error.

A detail worth noticing

For H_2 the 2-electron sector result and the all-sectors result are the **same number**. The chemically meaningful ground state happens to be the global ground state of the operator.

This is not guaranteed in general. For other molecules the global minimum can live in a sector with the wrong number of electrons, and an ansatz that does not conserve particle number can converge confidently onto a physically meaningless state. Phase 3 returns to this when choosing an ansatz.

Step 2.4 — Compare the three mappings

Mappings are unitary changes of basis, so they must all give the same spectrum. Confirming this is a good way to convince yourself the choice really is free.

Listing 2.4 — the mapping is a free choice

```

for mapping in ["jordan_wigner", "parity", "bravyi_kitaev"]:
    H, n = qubit_hamiltonian(0.735, mapping=mapping)
    M = qml.matrix(H, wire_order=range(n))
    print(f"{mapping:16s} qubits={n} terms={len(H.terms()[0]):3d} "
          f"ground state={np.linalg.eigvalsh(M)[0].real:.6f}")

```

```

jordan_wigner  qubits=4 terms= 15 ground state=-1.137306
parity         qubits=4 terms= 15 ground state=-1.137306
bravyi_kitaev  qubits=4 terms= 15 ground state=-1.137306

```

Step 2.5 — Taper the qubits away

Optional but genuinely instructive: exploit the symmetries and watch four qubits become one.

Listing 2.5 — symmetry tapering

```
def tapered_hamiltonian(r_angstrom=0.735):
    """Reduce the qubit count using the Hamiltonian's Z2 symmetries."""
    H, n = qubit_hamiltonian(r_angstrom)

    generators = qml.symmetry_generators(H)      # the conserved quantities
    paulix_ops = qml.paulix_ops(generators, n)   # the corresponding rotations
    sector = qml.qchem.optimal_sector(H, generators, ELECTRONS)

    H_tapered = qml.taper(H, generators, paulix_ops, sector)

    print(f"symmetry generators found {len(generators)}")
    print(f"optimal sector {sector}")
    print(f"qubits {n} -> {len(H_tapered.wires)}")
    print(f"terms {len(H.terms()[0])} -> {len(H_tapered.terms()[0])}")

    M = qml.matrix(H_tapered, wire_order=H_tapered.wires)
    print(f"ground state after tapering "
          f"{np.linalg.eigvalsh(M)[0].real:.8f} Ha")
    return H_tapered

symmetry generators found 3
optimal sector [1, -1, -1]
qubits 4 -> 1
terms 15 -> 3
ground state after tapering -1.13730604 Ha
```

Four qubits and fifteen terms become **one qubit and three terms**, with the ground-state energy unchanged to eight decimal places. The physics is identical; only the representation shrank. Having done this yourself, a hardcoded two-qubit Hamiltonian is no longer a mystery object — it is a specific point on a spectrum of choices you now control.

4. Traps in this phase

Trap	Symptom	Fix
Bohr vs Ångström	Absurd minimum position, or no minimum at all.	Convert explicitly in one place. $1 \text{ Å} = 1.8897261 \text{ Bohr}$. Check this first when anything looks wrong.
Comparing to the wrong sector	Exact diagonalisation disagrees with FCI by a large margin.	The qubit Hamiltonian spans all electron numbers. Project onto the correct particle-number sector, as Listing 2.3 does.
Assuming a tutorial Hamiltonian is comparable	Your energies differ from a reference by $\sim 0.72 \text{ Ha}$.	That is the nuclear repulsion term at equilibrium. Published 2-qubit H_2 operators are frequently electronic-only.
Wire-order confusion	<code>qml.matrix</code> gives a matrix that does not match hand calculations.	Always pass <code>wire_order=</code> explicitly. Qubit-ordering conventions differ between frameworks — Qiskit reads Pauli strings right-to-left, PennyLane left-to-right.
Rebuilding the Hamiltonian outside the loop	Curve is smooth but flat, or the minimum sits exactly at your first grid point.	Covered fully in Phase 4 — but note now that <code>qubit_hamiltonian</code> takes <code>r</code> as an argument for exactly this reason.

5. What was done in this phase

- **Built the translation layer** from nuclear geometry to qubit operator, parameterised by bond length — which is what makes Phase 4 possible at all.
- **Learned second quantisation**: the Hamiltonian is written in creation and annihilation operators, so a mapping translates *operators*, not states.
- **Saw the naive encoding fail** numerically, in eight lines, and understood that the failure is about signs rather than states.
- **Understood Jordan-Wigner** as a Z string that charges one minus sign per electron hopped over, and the locality cost that comes with it.
- **Understood parity mapping** as its dual, and why it — not Jordan-Wigner — is what enables the two-qubit reduction behind every tutorial Hamiltonian.
- **Verified the qubit Hamiltonian against Phase 1's FCI** to $1.3 \times 10^{-15} \text{ Ha}$. This is the gate: Phase 3 is not worth starting until this assertion passes.
- **Confirmed the mapping is a free choice** — all three give identical spectra — and **tapered four qubits down to one**, turning an inherited black box into a controlled decision.

6. Checklist before moving on

1. `hamiltonian.py` returns a 4-qubit, 15-term operator at 0.735 Å .
2. Listing 2.3's assertion passes: exact diagonalisation of the qubit Hamiltonian reproduces PySCF's FCI to better than 10^{-8} Ha .
3. You have run the same check at a second geometry, say 1.5 Å , and it still passes.

4. All three mappings give the same ground-state energy.
5. You can explain why a mapping is needed at all, without using the word "convention".

Next

Phase 3 — Single-point VQE. Everything up to now has been setup. Phase 3 finally runs the quantum algorithm, and Phase 2's verification is what lets you tell whether the answer is right.